

Multi-Agent F1 Race Strategy

Final plan: what we build, how the app works, every command, and how we prepare and defend each choice

Course component	Case study (20 marks): Presentation 1, Problem Definition (10) + Presentation 2, Implementation & Demo (10). Maximum 6 slides.
What we build	A 20-lap simulated race at Silverstone with 12 agents. Strategy agents plan and change pit stops; the examiner can trigger a Safety Car or rain during the race.
Teams and agents	McLaren (NOR, PIA) · Ferrari (LEC, HAM) · Red Bull (HAD, VER) · Mercedes (RUS, ANT). Each team has 1 strategist agent and 2 driver agents. Race Control is part of the environment.
Algorithms	From Russell & Norvig, AI: A Modern Approach (AIMA): A* search (Ch. 3), belief-state update (Ch. 4), minimax with alpha-beta (Ch. 5), condition-action rules (Ch. 2), replanning after events
Data	The real 2026 British Grand Prix (5 July 2026, dry), prepared once and stored inside the project
App	Python backend with a terminal for commands and the full conversation log; a Next.js page with one panel: the track map
Project folder	Desktop/F1-Strategy-MAS , independent of every other project; data already prepared

Contents

1. The project on one page	4
1.1 What the examiner sees	4
1.2 What we must prove, and where	4
1.3 Rubric traceability	4
2. F1 in 60 seconds (for the audience)	6
3. Problem statement	6
4. PEAS formulation	7
4.1 System-level PEAS	7
4.2 PEAS per agent type	7
4.3 Performance measure	7
4.4 Global impact matrix: how one agent's action changes the environment for others	7
5. Environment and agent analysis	8
5.1 Environment classification (AIMA Ch. 2)	8
5.2 Multi-agent dynamics	8
5.3 State-space size (why the problem must be decomposed)	8
5.4 Communication bottlenecks and friction points	9
6. Algorithms and how we defend them	9
6.1 A* for the pit-stop plan	10
6.2 Belief states for hidden rival tyres	11
6.3 Minimax with alpha-beta for pit duels (undercut and overcut)	11
6.4 Drivers: condition-action rules	12
6.5 Sudden events: replanning	12
6.6 One-sentence defences	13
7. Agent interactions	13
7.1 Messages	13
7.2 Request → reply, and the strategist has the final say	13
7.3 The 'Decision changed' record: proof that interaction happened	14
7.4 Interaction 1: a driver changes the strategist's decision	14
7.5 Interaction 2: a rival team's move (undercut and overcut)	15
7.6 Interaction 3: Safety Car	15
7.7 Interaction 4: rain	16
8. Race data: the 2026 British Grand Prix	17
8.1 Why this race	17
8.2 How the data was prepared (done once)	17
8.3 Files in data/	17
8.4 The race model and where each number comes from	18
9. How the app works	19
9.1 Starting it	19
9.2 What happens during the race	19

9.3 The page: one panel, the track map	20
9.4 The terminal.....	21
10. Commands.....	21
11. Demo script (about 7 minutes).....	22
12. Tools and project structure	22
12.1 Tools.....	22
12.2 Folder structure	23
13. Build plan	23
14. Tests.....	24
15. Preparing and defending	24
15.1 Slides (6).....	24
15.2 Preparation checklist	24
15.3 Q&A	25
16. Later: merging into the F1 app	25

1. The project on one page

We simulate a Formula 1 race in which each team is run by agents. A **strategist agent** decides when its cars stop for new tyres, and two **driver agents** drive and report how their tyres feel. Teams cooperate internally and compete with each other. Strategists never see rival teams' tyre condition; they only see a public timing screen. During the race the examiner can type **safety_car** or **rain** into a terminal, and every team has to rethink its plan.

1.1 What the examiner sees

- A full-screen map of Silverstone with 8 cars moving, in 2026 team colours.
- Short notifications on the map whenever an agent sends a message: driver reports, strategist instructions, pit-lane entries.
- A red '**Decision changed**' card whenever a strategist changes a plan, saying **before** → **after** → **because of which agent or event**.
- A terminal that prints every message in colour and answers commands such as **plan NOR** (shows the A* search) and **why ANT** (explains the last change).
- A Safety Car or rain arriving at a moment the examiner chooses, with all four teams reacting.

1.2 What we must prove, and where

We must show that we understand...	Proved by
The problem as an AI problem (PEAS, environment type, multi-agent nature)	§3–5 and slides 1–2
Which search algorithms fit which decision, and why the others do not	§6 with a worked A* example, a minimax tree and comparison tables; plan and why commands live
Real interaction: agents changing each other's decisions	§7 and the 'Decision changed' cards; the end-of-race summary counts changes by cause
A working, well-organised implementation on real data	§8–13: real 2026 race data, one-command start, clean folder structure, tests

One rule for the whole demo

Every moment on screen should answer two questions: **who told what to whom**, and **whose decision changed because of it**.

1.3 Rubric traceability

Rubric criterion (weight)	Covered in	Seen in the demo at
P1 · PEAS (3)	§4 PEAS tables, performance measure, global impact matrix	Scene 1: plans made from driver reports and the timing screen
P1 · Environment & agents (3)	§5 classification, cooperation vs competition, state-space size, bottlenecks	Scene 3: rival tyres are hidden and only guessed
P1 · Algorithms & search (3)	§6 A*, belief states, minimax, with rejected alternatives	Scene 1: plan NOR ; scene 3: why ANT
P1 · Q&A (1)	§15.3 prepared answers	n/a
P2 · Tools (3)	§12 tools and reasons; §9 how the app runs	Scene 0: backend and page start with one command each

Rubric criterion (weight)	Covered in	Seen in the demo at
P2 · Interaction (3)	§7 messages, request → reply, decision-change record, 4 interactions	Scenes 2–5: decision cards naming their cause
P2 · Demo & testing (3)	§11 script; §14 tests	Scenes 4–5: examiner chooses when the Safety Car and rain happen
P2 · Code structure (1)	§12.2 folder tree; §14 tests	Slide 4

PART A — THE AI PROBLEM

2. F1 in 60 seconds (for the audience)

Most of the audience will not follow F1. Put this table on slide 1 and use the same plain words everywhere in the app.

Term	Plain meaning	Why it matters to our agents
Race	20 laps; the car with the least total time wins	The goal every team plans for
Tyres	Soft : fastest but wears out quickly · Medium : in between · Hard : slowest but lasts longest · Wet : only useful in rain	The main choice the strategist makes
Tyre wear	The older the tyre, the slower each lap	Sooner or later the car must stop
Pit stop	Drive through the pit lane to get new tyres; costs about 21 seconds (measured from the real race)	The main action; the key question is when
Tyre rule	Each car must use two different dry tyre types, unless it has used Wet tyres	Every car stops at least once
Undercut	Stop before your rival. Your new tyres are faster, so you can come out ahead	One team's move forces the other to react
Overcut	Stay out while your rival stops, if they would come out behind slower cars	The opposite answer to the same situation
Safety Car	After a crash everyone drives slowly behind a safety car. A stop now costs only about 10 seconds	Sudden change: every team replans at once
Rain	Dry tyres become very slow on a wet track; Wet tyres are needed	Sudden change: when should each car switch?
Strategist / Driver	Strategist: the team's planner at the pit wall. Driver: in the car, feels the tyres	Our two agent types, with different information

3. Problem statement

An F1 race is a sequence of decisions made with incomplete information. Each team must decide **when to stop and which tyres to fit** while rival teams make the same decisions, hide their tyre condition, and the race can change suddenly (Safety Car, rain). Drivers feel things the strategist cannot see, and the strategist sees things the drivers cannot.

We model this as a **hierarchical multi-agent system**. In each team, a strategist agent plans with search and two driver agents drive and report. Teams **cooperate internally** and **compete with each other**. Agents affect each other through **messages** (team radio, the public timing screen) and through the **shared track** (blocking, where a car rejoins after a stop).

Goal

Show, in a way anyone can follow, that 12 agents with partial information **plan using search, communicate, and change their decisions because of other agents and events**, with every change traceable on screen.

4. PEAS formulation

4.1 System-level PEAS

	Description
Performance (P)	Team points from both cars (8-car scale: 25, 18, 15, 12, 10, 8, 6, 4). Secondary: total race time; decision changes that gained or kept a position.
Environment (E)	Static: Silverstone (5.8 km, 18 corners, 3 long straights where overtaking is possible), pit lane with one box per team, tyre types {Soft, Medium, Hard, Wet}, 20 laps, two-dry-tyre rule. Dynamic: car positions and gaps, tyre age and wear, weather (dry/rain), Safety Car on/off, other teams' plans.
Actuators (A)	Strategist: INSTRUCTION 'pit now for tyre X' or 'stay out'; pace 'push / normal / save tyres'. Driver: pace mode, overtaking attempt on a straight, entering the pit lane; messages INFO (report), REQUEST ('box now?'), REPLY ('OK' / 'tyres are fine').
Sensors (S)	Driver: own tyre wear, lap time, gap to the car ahead and behind, grip (rain). Strategist: team radio from its own drivers; public timing screen (positions, gaps, lap times, tyre type, laps since last stop); Race Control announcements. Hidden: rivals' tyre wear and rivals' plans.

4.2 PEAS per agent type

	Strategist agent (×4)	Driver agent (×8)
Performance	Least predicted race time for each car; win position duels against close rivals	Follow the plan, keep the tyres alive, gain places on the straights
Environment	Whole race, seen only through reports and the timing screen	Own car and the cars just ahead and behind
Actuators	Instructions to its own two drivers	Pace mode, overtaking attempt, pit entry, messages to its strategist
Sensors	Team radio + timing screen + Race Control	Own car sensors + what it can see around it
AIMA agent type (Ch. 2)	Goal- and utility-based agent that plans with search	Model-based reflex agent (condition-action rules)

4.3 Performance measure

Plan cost (A^* path cost, lower is better):

$T(\text{plan}) = \text{sum of predicted lap times} + \text{sum of pit-stop losses}$

$\text{lap time} = \text{base} + \text{tyre offset} + \text{tyre wear} \times \text{tyre age} + \text{fuel effect} \times \text{lap} \quad (+ \text{rain, traffic})$

Duel value (minimax, higher is better for us):

$V = \text{our predicted gap to the rival after both cars have stopped (seconds)}$

Team result (reported at the end):

$U_{\text{team}} = \text{points}(\text{car 1}) + \text{points}(\text{car 2})$

4.4 Global impact matrix: how one agent's action changes the environment for others

Action (by whom)	Effect on own car	Effect on everyone else	Who has to react
Pit stop (strategist → driver)	Loses ~21 s, gets new faster tyres	Car rejoins in a new place; can drop in front of a rival or behind traffic	Nearby rivals' strategists (undercut / overcut)
Push (driver)	Faster now, tyres wear sooner	Closes the gap to the car ahead	Car ahead, own strategist (earlier stop)

Action (by whom)	Effect on own car	Effect on everyone else	Who has to react
Save tyres (driver)	Slower now, tyres last longer	Car behind catches up	Car behind
Blocking (automatic on track)	Holds position	Car behind loses 0.3 s per lap until a straight	Following driver reports 'stuck'; strategist may change plan
Safety Car (Race Control)	Everyone slows down	Stops become cheap for all teams at once	All 4 strategists at the same moment
Rain (Race Control)	Dry tyres become slow	Order changes depending on who switches first	All drivers report; all strategists decide

5. Environment and agent analysis

5.1 Environment classification (AIMA Ch. 2)

Property	Our environment	Why
Observable?	Partially observable	Strategists see rivals only through the timing screen. Other teams' tyre wear and plans are hidden.
Agents	Multi-agent: cooperative and competitive	Teammates share points (cooperative); teams fight for positions (competitive).
Deterministic?	Stochastic	Lap times vary slightly, overtakes can fail, Safety Car and rain arrive unexpectedly.
Episodic?	Sequential	Stopping now changes every later lap.
Static?	Dynamic	The race keeps moving while agents decide.
Discrete?	Continuous, discretised by lap	Cars move continuously; planning happens once per lap.
Known?	Known rules, unknown rival details	Lap-time rules are known; rival tyre condition and event timing are not.

5.2 Multi-agent dynamics

- **Cooperative (inside a team):** drivers report to their strategist; teammates share one pit box, so their stops must be coordinated.
- **Competitive (between teams):** a position gained by one team is lost by another. Undercut and overcut are moves in this competition.
- **Hierarchy:** the strategist decides; a driver can answer once ('tyres are fine'), then the strategist has the final say. No two agents ever give opposite orders.
- **Indirect interaction:** rival agents never message each other. They react to what they see on the public timing screen and on track.

5.3 State-space size (why the problem must be decomposed)

Formulation	Size	What it means
One car, each lap choose {stay, pit Soft, pit Medium, pit Hard}, 20 laps	$4^{20} \approx 1.1 \times 10^{12}$ action sequences	Listing every plan is impossible
Same problem as a state graph: lap $\times$ tyre $\times$ tyre age $\times$ tyres used $\times$ stops	$\leq 21 \times 3 \times 21 \times 7 \times 4 = 37,044$ states	Small enough for A*

Formulation	Size	What it means
All 8 cars planned together	$(4^{20})^8 = 4^{160} \approx 2 \times 10^{96}$	Joint planning is hopeless, so each strategist plans its own cars
Minimax over the whole race (4 teams, 16 choices each per lap)	$16^4 = 65,536$ branches per lap, 20 laps deep	Minimax only fits small two-player duels

5.4 Communication bottlenecks and friction points

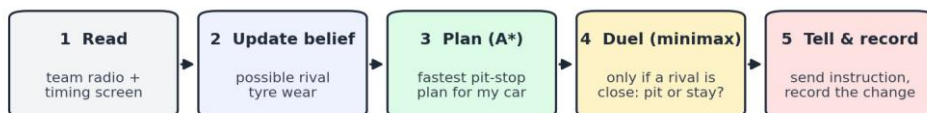
- **One decision maker per team:** simple and consistent, but every decision waits for the strategist.
- **Delayed, partial information:** the timing screen shows gaps and tyre type, not wear. A rival's stop is only known once the car enters the pit lane.
- **Shared pit box:** two teammates cannot stop at the same moment without the second one waiting.
- **Undercut window:** if the gap between two cars is smaller than what new tyres gain, whoever stops first wins. Both teams know this, which makes it a game.
- **Outside our scope (discussion only):** real teams also deal with radio loss, sensor failure and team orders. We leave them out to keep the demo clear.

6. Algorithms and how we defend them

Each decision in the race is a different **type of problem**, so it uses the AIMA algorithm designed for that type. This section is the main talking point for 'why this algorithm and not another'.

Decision	Problem type	AIMA chapter	Algorithm we use
When to stop and which tyres (own car)	Single-agent path finding with different step costs	Ch. 3 Solving problems by searching	A* search with an admissible heuristic
How worn are the rival's tyres?	Partially observable state	Ch. 4 Search in complex environments (belief states)	Belief-state update (predict, then filter by observation)
Stop now or stay out against a close rival?	Two-player, zero-sum game	Ch. 5 Adversarial search	Minimax with alpha-beta pruning , depth 2, evaluation function at the cutoff
How to drive right now	Fast local reaction	Ch. 2 Intelligent agents	Condition-action rules (model-based reflex agent)
Safety Car or rain arrives	The world changed after planning	Ch. 4 online search idea: act, observe, replan	Replan with A* from the new state

STRATEGIST (one per team) — runs at the end of every lap and whenever a message or event arrives



DRIVER (two per team) — simple rules every moment, talks once per lap or when something changes



Figure 1. What each agent does, and where each algorithm is used

6.1 A* for the pit-stop plan

Element	Definition
State	(lap, current tyre, tyre age, tyre types used so far, stops made)
Start state	The car's state now (for a rival's car, the believed state)
Actions	STAY OUT, or PIT for Soft / Medium / Hard (plus Wet once it is raining); at most 3 stops
Step cost	Predicted lap time; a PIT action adds the pit loss (21.0 s, or 10 s under the Safety Car)
Goal test	Lap 20 reached and at least two dry tyre types used (or a Wet tyre used)
Heuristic $h(n)$	(laps left) $\times$ (fastest possible lap time) + (one pit loss if a second tyre type is still required)

Why the heuristic is admissible (AIMA: heuristics from relaxed problems)

h is the exact cost of an **easier (relaxed) problem**: tyres never wear out, the fastest tyre is always fitted, and only one more stop is needed. A relaxed problem can never cost more than the real one, so **h never overestimates**. A* with an admissible heuristic returns the cheapest plan.

It is also **consistent**: one more lap lowers h by exactly the fastest lap time, and no real lap is faster than that. A* therefore never needs to re-open a state.

Worked example (numbers from data/race_model.json): 3 laps left, Medium tyres 8 laps old, a second tyre type is still required

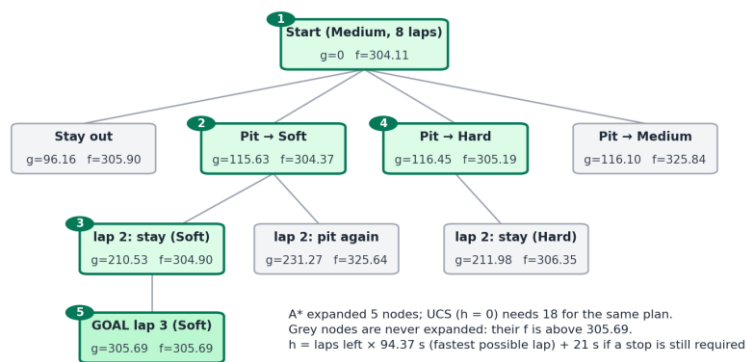


Figure 2. A* on a small case using the real model numbers (checked in code: A* expands 5 nodes, UCS 18, same plan)

Why not the other search algorithms?

Algorithm (AIMA)	Why not for the pit plan	Verdict
Breadth-first search	Finds the plan with the fewest steps, not the least time. A stop (~21 s) and a lap (~95 s) cost different amounts, so it would pick wrong plans	Rejected
Depth-first, depth-limited, iterative deepening	Not cost-optimal when step costs differ; DFS can commit to a bad early choice	Rejected
Uniform-cost search (Dijkstra)	Always finds the cheapest plan, but explores blindly in every direction	Kept as the comparison in the plan command
Greedy best-first	Fast, but only follows h , so it can miss the cheapest plan	Rejected (not optimal)
A*	Cheapest plan, like UCS, but guided by h , so far fewer nodes are expanded	Chosen
IDA*, RBFS, SMA*	Memory-saving versions of A*; our 37,044 states fit in memory easily	Not needed
Bidirectional search	There are many goal states (any tyre, any age at lap 20), so searching backwards is awkward	Rejected

Algorithm (AIMA)	Why not for the pit plan	Verdict
Hill climbing, simulated annealing, genetic algorithms	Improve a full plan step by step, but can get stuck and give no guarantee of the best plan. They suit huge spaces; ours is small	Rejected
CSP backtracking (Ch. 6)	Finds a plan that satisfies the rules, not the fastest one; the rules are already our goal test	Rejected
AND-OR search (plans for every possible event)	A Safety Car or rain could come on any lap, so a conditional plan explodes (2^{20} branches). Replanning when the event happens is simpler	Rejected; we replan instead

6.2 Belief states for hidden rival tyres

A strategist cannot see a rival's tyre wear. Following AIMA Ch. 4 (search in partially observable environments), it keeps a **belief state**: the set of wear levels the rival could still have.

Belief for each rival car: { LOW wear (x0.8), NORMAL wear (x1.0), HIGH wear (x1.25) }

Every lap:

1. Predict: for each level still in the belief, predict the rival's lap time
2. Update: keep only the levels whose prediction is within 0.4 s of the real lap time (skip the update on laps behind the Safety Car or stuck in traffic)
3. Use: 'rival may stop soon' if ANY remaining level says its tyres are past 70% worn

Alternative	Why not
Assume every rival is average	The strategist would be surprised by every undercut from a rival with high wear
Full probability model (Bayes, hidden Markov model)	More precise, but needs probability distributions and is outside the search portion. A set of possible levels is enough to decide 'could they stop soon?'
Let strategists read rivals' true wear	Breaks partial observability, which is the point of the environment analysis

6.3 Minimax with alpha-beta for pit duels (undercut and overcut)

When a rival car is within about 3 seconds and neither car has stopped yet, the decision is no longer just 'what is fastest for me'. It depends on what the rival does. For those two cars the position is **zero-sum** (one gains exactly what the other loses), which is the setting minimax is designed for.

Element	Definition
Players	Our strategist (MAX) and the rival strategist (MIN)
Moves	PIT now or STAY out, for each player
Turn order	We move first and the rival replies. It sees our stop on the timing screen, so this is the cautious assumption
Depth	2 plies (our move, their reply), repeated every lap
Evaluation at cutoff	Predicted gap after both cars have stopped, using the same lap-time model as A* (AIMA: heuristic minimax with an evaluation function)
Result	PIT wins → undercut . STAY wins (for example the rival would rejoin behind slower cars) → overcut
After the rival has already stopped	Its move is known, so its branch is fixed and we only compare our two options with the same evaluation

Pit duel: Mercedes (MAX) chooses, Ferrari (MIN) replies. Value = Antonelli's gap to Leclerc after both stops (s)

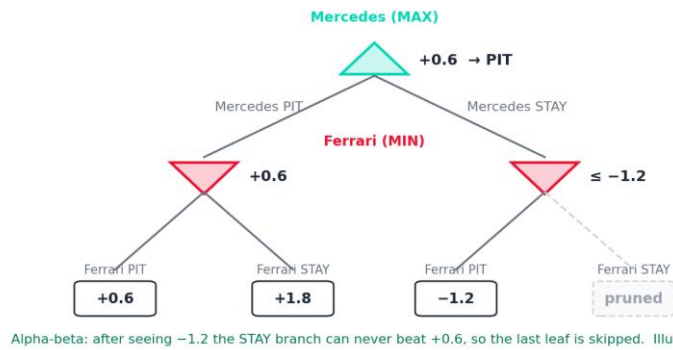


Figure 3. Minimax with alpha-beta: Mercedes should stop Antonelli now; the last leaf never needs to be evaluated

Why not the other game-search algorithms?

Algorithm (AIMA)	Why not	Verdict
Minimax over the whole race	4 teams, not 2; teammates are not enemies; 65,536 joint moves per lap for 20 laps	Rejected (used only for 2-car duels)
Minimax + alpha-beta, depth-limited	Exactly two players, one position at stake, 4 leaves; alpha-beta skips leaves that cannot matter	Chosen
Expectiminimax (chance nodes)	Would add a chance node for a Safety Car on every lap. We replan when a Safety Car actually happens, which keeps the tree tiny	Not needed (good Q&A point)
Monte Carlo tree search	Built for huge game trees (Go); ours has 4 leaves	Not needed

6.4 Drivers: condition-action rules

Drivers react many times a second, so they do not search. They are **model-based reflex agents** (AIMA Ch. 2): they keep a small internal state (tyre wear, weather, gap ahead) and apply rules in priority order.

If...	then the driver...
An instruction arrives	follows it and REPLIES 'OK'; if told to stop early while its tyres are under 50% worn, it REPLIES 'tyres are fine, I can stay out' once
Tyre wear passes 70%	sends INFO 'tyres worn' and a REQUEST 'box now?'
It is raining and the car is on dry tyres	sends INFO 'track slippery' and a REQUEST 'box for wets?'
It has been stuck within 1 s of a slower car for 2 laps	sends INFO 'stuck behind [car]'
It is on one of the 3 long straights, within 1 s of the car ahead and at least 0.5 s per lap faster	attempts to overtake: succeeds with probability 0.5 (seeded random); otherwise stays behind
Otherwise	drives at the pace mode the strategist set

6.5 Sudden events: replanning

A Safety Car or rain does not break the plan; it **changes the costs**. Under a Safety Car a stop costs about 10 s instead of 21 s. In rain a dry tyre is about 6 s slower per lap and Wet tyres become an action. The strategist runs A* again from the current state with the new costs. This 'act, observe, replan' loop is the online-search idea from AIMA Ch. 4, and it is why we do not need AND-OR search or Expectiminimax.

Trigger	Who replans	Algorithm run
End of each lap	Own strategist	Belief update, A*

Trigger	Who replans	Algorithm run
Driver report or reply (worn tyres, 'tyres are fine', slippery track, stuck)	Own strategist	A* with the reported condition
Rival within 3 s and neither has stopped / rival has just stopped	Both strategists	Minimax
Safety Car or rain announced	All 4 strategists at once	A* with the new costs

6.6 One-sentence defences

Choice	Say this
A*	Different actions cost different times, so we need a cost-optimal search, and A* with a relaxed-problem heuristic finds the same best plan as UCS while expanding far fewer nodes.
Belief states	Rival tyres are hidden, so we keep the set of wear levels that still match the lap times we observe, which is how AIMA handles partial observability without probabilities.
Minimax + alpha-beta	A pit duel between two cars is a small two-player zero-sum game, so minimax picks the move that is best even if the rival answers perfectly, and alpha-beta skips branches that cannot change the answer.
Reflex drivers	Drivers must react instantly and locally, so rules are enough; the planning happens at the strategist.
Replanning	Safety Car and rain only change costs, so re-running A* from the new state is simpler than planning for every possible event in advance.

7. Agent interactions

7.1 Messages

Agents communicate with five plain message types. They correspond to FIPA-ACL speech acts (INFORM, REQUEST, AGREE), which is worth one sentence in the presentation.

Type	Meaning	Example
INFO	Report a fact	Hamilton → Ferrari strategist: 'Tyres 70% worn, losing half a second a lap'
REQUEST	Ask for a decision	Norris → McLaren strategist: 'Box this lap?'
INSTRUCTION	A decision, sent down	Mercedes strategist → Antonelli: 'Stop now to cover Leclerc'
REPLY	Answer to an instruction	Antonelli → Mercedes strategist: 'OK, boxing' / 'Tyres are fine, I can stay out'
EVENT	Public news from Race Control or the timing screen	'Safety Car deployed' · 'Leclerc entered the pit lane'

Who can read what: team radio (INFO, REQUEST, INSTRUCTION, REPLY) is delivered only inside the team. EVENTS are public. The screen and terminal show everything so the audience can follow, but each agent only receives what it is allowed to see.

7.2 Request → reply, and the strategist has the final say

```

Strategist —INSTRUCTION→ Driver      'stop on lap 9'
Driver      —REPLY→          Strategist 'OK' → done
                                   or 'tyres are fine, I can stay out' (allowed once)
Strategist re-runs A* with the driver's information and sends a final INSTRUCTION → no further replies

Driver      —REQUEST→         Strategist 'box now?'
Strategist re-runs A* and answers with an INSTRUCTION (box now / stay out)

```

Because the strategist always ends the exchange, there is no endless back-and-forth and no deadlock. Teammates sharing the pit box follow a simple rule: the car ahead on track stops first and the second car stops right behind it, unless A* says waiting a lap is cheaper.

7.3 The 'Decision changed' record: proof that interaction happened

Whenever a strategist's new plan for a car differs from the old one, it writes a record. The page shows it as a red card, the terminal prints it, and **why** <driver> explains it.

```
DecisionChange(
  lap    = 9,
  team   = 'Mercedes',  car = 'ANT',
  before = 'stop on lap 11 (Hard)',
  after  = 'stop now, lap 9 (Hard)',
  cause  = message #41 'Leclerc entered the pit lane' (from Ferrari, via timing screen),
  reason = 'Minimax: stay out -> 1.2 s behind; stop now -> 0.6 s ahead',
)
```

A change is only recorded when there is a cause: a message, an event, or a rival's move. At the end, a summary counts decision changes by cause (driver reports, rivals, Safety Car, rain), which gives a one-line answer to 'did the agents really interact?'.

7.4 Interaction 1: a driver changes the strategist's decision

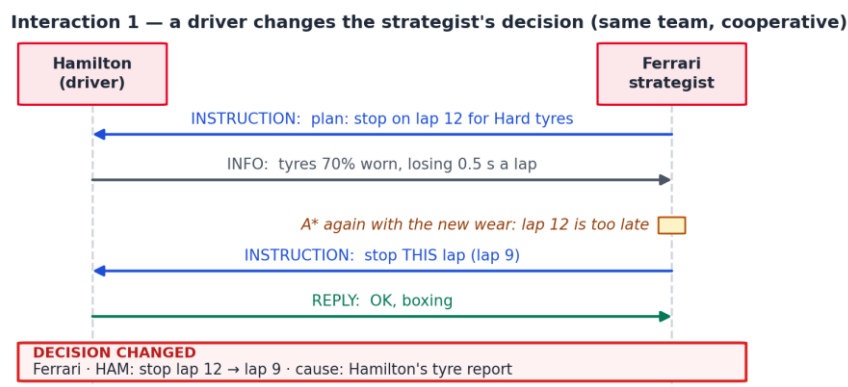


Figure 4. Cooperative interaction inside a team

Seen on screen: Hamilton's message appears as a notification, then a red card: 'Ferrari · HAM: stop lap 12 → lap 9 · because: Hamilton's tyre report'. The reverse also happens: a driver replies 'tyres are fine' and the strategist moves the stop later.

7.5 Interaction 2: a rival team's move (undercut and overcut)

Interaction 2 — a rival team's move changes the plan (different teams, competitive)

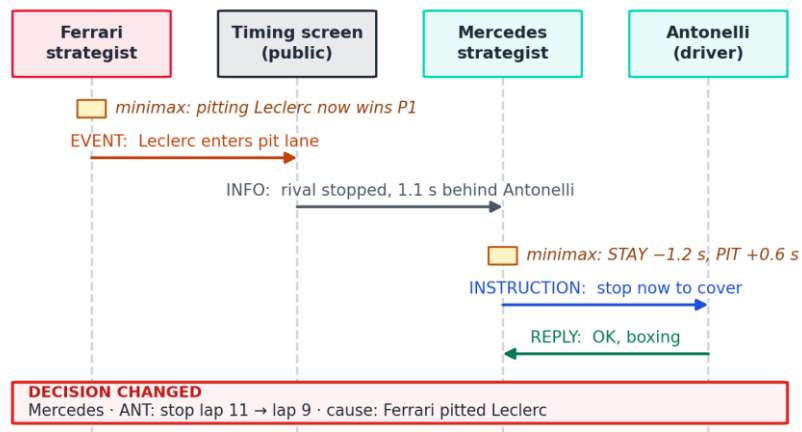


Figure 5. Competitive interaction between teams through the public timing screen

Seen on screen: Leclerc's car gets an 'IN PIT' ring on the map, a ring pulses on Antonelli, and the card says 'because: Ferrari stopped Leclerc'. Typing **why ANT** prints the two minimax values. In another situation the same check returns STAY (overcut), and the card reads 'stay out: Leclerc will rejoin behind Norris'.

7.6 Interaction 3: Safety Car

Interaction 3 — Safety Car: every team re-plans at once; teammates share one pit box

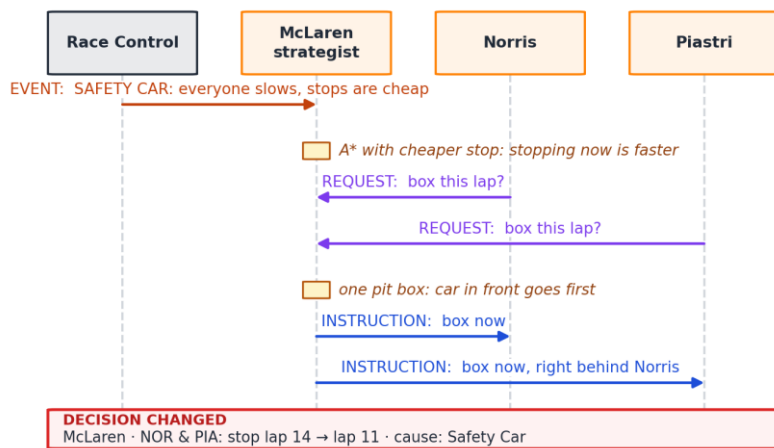


Figure 6. One event, four teams replanning at the same time, teammates sharing a pit box

7.7 Interaction 4: rain

Interaction 4 — Rain: the driver's report decides when to switch to wet tyres

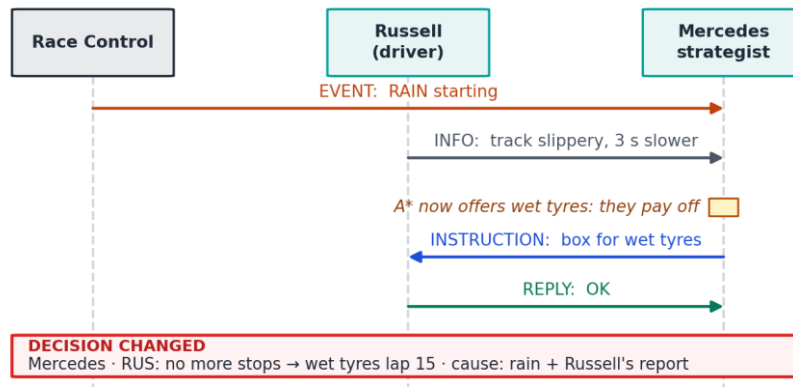


Figure 7. The driver's report plus the event lead to the tyre switch

Seen on screen: the map turns blue-tinted, drivers report slippery conditions, and teams switch to Wet tyres on **different laps**. The order changes, and each card explains why that team switched when it did.

PART B — DATA AND THE APP

8. Race data: the 2026 British Grand Prix

8.1 Why this race

Fact (from the data)	Why it helps
Held 5 July 2026 at Silverstone, 52 laps, 5,817 m per lap	The most recent race at our track
Dry: no rainfall readings, air 24.8 °C, track 40.0 °C; only Soft, Medium and Hard tyres used	A clean dry race to start from; rain only happens when we type rain
Our 8 drivers started from grid slots 1–8: ANT, LEC, HAM, RUS, HAD, NOR, VER, PIA	We use the real starting order
Real result among our 8: LEC 1st, RUS 2nd, HAM 3rd, NOR 4th, HAD 5th, PIA 11th, ANT 15th, VER retired	Good closing slide: compare with our simulated result
Safety Car on the last laps; 5 of our 8 cars stopped on laps 47–48 , right as it came out	Real-world proof that a Safety Car makes every team rethink at once
First stops were spread out: VER lap 17, HAD 19, HAM and RUS 23, LEC 25, NOR 28, ANT 35 (PIA stopped on lap 2)	Real teams chose very different stop timing, which is exactly the decision our strategists make

8.2 How the data was prepared (done once)

- FastF1 3.8.3 downloaded the race (laps, telemetry, weather, flags), reusing the cache folder of the F1-Telemetry-Analysis project so nothing was downloaded twice.
- `tools/extract_race_data.py` wrote the small CSV and JSON files below into `data/`. The racing line comes from the fastest race lap, drawn the same way as the existing track map.
- Corner positions normally come from the MultiViewer circuit service, which was unreachable from our network. The script instead lined up the 2026 lap with a saved Silverstone layout (rotation 92°, average distance between the two lines 3.0 m, well inside the track width) and placed the 18 corners on the 2026 lap.
- `tools/fit_model.py` measured tyre wear, fuel effect and pit-stop loss from the laps and wrote `race_model.json`.

The app never touches FastF1, the internet or the other project

Everything the app needs is in `data/` (about 120 KB). A teammate only needs this folder. The tools are kept so the data can be regenerated or another race used later.

8.3 Files in data/

File	Contents	Size
<code>silverstone_2026_track.csv</code>	Racing line: x, y, distance	37.9 KB
<code>silverstone_2026_corners.csv</code>	18 corners with label positions; the 3 long straights (T5–T6 732 m, T14–T15 865 m, T18–T1 592 m) are computed from their gaps	1.8 KB
<code>silverstone_2026_laps.csv</code>	Every lap of all 22 drivers: lap time, tyre, tyre age, stint, pit in/out, flag status, position, 'clean lap' flag	74.6 KB
<code>silverstone_2026_summary.json</code>	Race facts used above	0.9 KB
<code>teams.json</code>	The 4 teams, 2026 colours, drivers, real grid and finish, hidden wear level per driver	1.7 KB

File	Contents	Size
race_model.json	Every number the simulator uses, each tagged 'data' or 'hand' (next section)	2 KB

8.4 The race model and where each number comes from

The demo race is **20 laps instead of 52**. To keep strategy meaningful, tyre wear and fuel effect are multiplied by $52/20 = 2.6$, so a 20-lap race wears tyres like the full race.

Quantity	Value used	Source	Reason
Base lap time	94.97 s	Data	Median driver pace on 818 clean laps
Fuel effect	-0.167 s per lap (real -0.064, $\times 2.6$)	Data	Cars get lighter and faster
Medium tyre	offset 0, wear 0.132 s per lap of age (real 0.051, $\times 2.6$), life 12 laps	Data	440 clean Medium laps
Soft tyre	-0.6 s, wear 0.264, life 7 laps	Hand	Only 14 clean Soft laps: faster than Medium, wears about 2 $\times$
Hard tyre	+0.4 s, wear 0.079, life 18 laps	Hand	Medium and Hard pace cannot be separated in this race: slower, wears about 0.6 $\times$
Pit stop loss	21.0 s	Data	Median of 24 green-flag stops (15 of them took 18–23 s)
Safety Car	3 laps, lap times $\times 1.4$, stop costs 10 s, no overtaking	Hand	Standard behaviour
Rain	Dry tyres +6 s per lap; Wet tyres +2.5 s; Wet tyres on a dry track +3 s	Hand	The real race was dry
Traffic	Within 1 s of the car ahead: +0.3 s per lap; overtaking only on the 3 straights	Hand	Simple and visible
Randomness	Lap-time noise 0.15 s; overtaking chance; fixed by a seed	Hand	Repeatable demo

How to defend the numbers

We use the real race where it gives reliable numbers (base pace, fuel, Medium wear, pit loss) and **label every hand-set value with its reason**. The project is about how agents decide and interact, not about perfect tyre physics, and every agent uses the same model, so the comparison between their decisions is fair.

9. How the app works

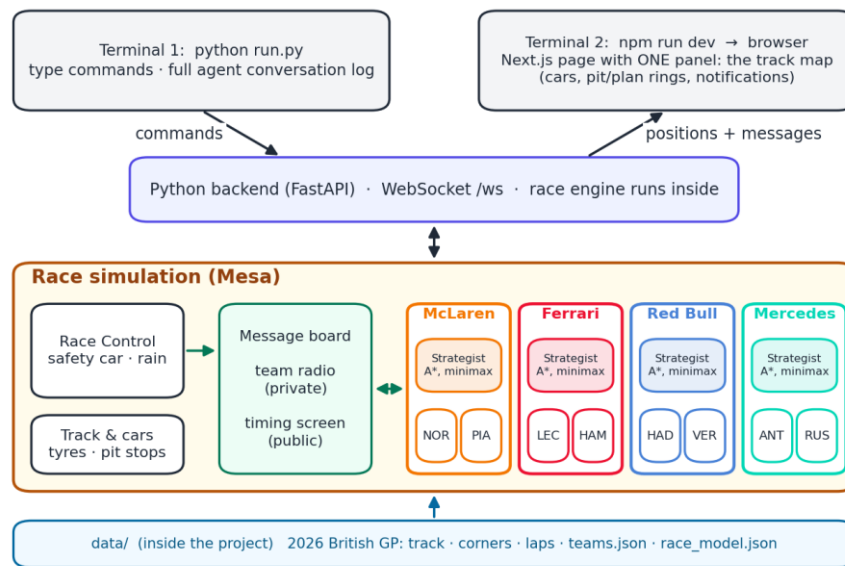


Figure 8. The whole system: Python backend with the race inside, a terminal, and one Next.js page

9.1 Starting it

```
Terminal 1 (backend + commands):
pip install -r requirements.txt
python run.py                                # backend on http://localhost:8000, then a '>' prompt

Terminal 2 (the page):
cd web
npm install                                  # first time only
npm run dev                                  # open http://localhost:3000
```

9.2 What happens during the race

The race engine updates 10 times per simulated second and speeds up with the **speed** command. At the start all cars line up in the real 2026 grid order on Medium tyres, as in the real race, and each strategist runs A* to make its first plan. After that, the loop below repeats:

Step	What happens
1. Race Control	Applies any Safety Car or rain the examiner has triggered and announces it as a public EVENT
2. Cars move	Lap times come from the race model; traffic slows cars; overtakes happen on the 3 straights; pit stops take the pit loss
3. Drivers act	Each driver applies its rules and sends INFO, REQUEST or REPLY messages
4. Messages delivered	Team radio only to the team; timing screen and Race Control to everyone
5. Strategists decide	At each lap end or when a message arrives: belief update → A* → minimax if a rival is within 3 s → compare with the current plan → send INSTRUCTION and record any DecisionChange
6. Screen updated	The backend streams car positions (10 per second), messages and decision changes to the page; the terminal prints each message in colour

9.3 The page: one panel, the track map

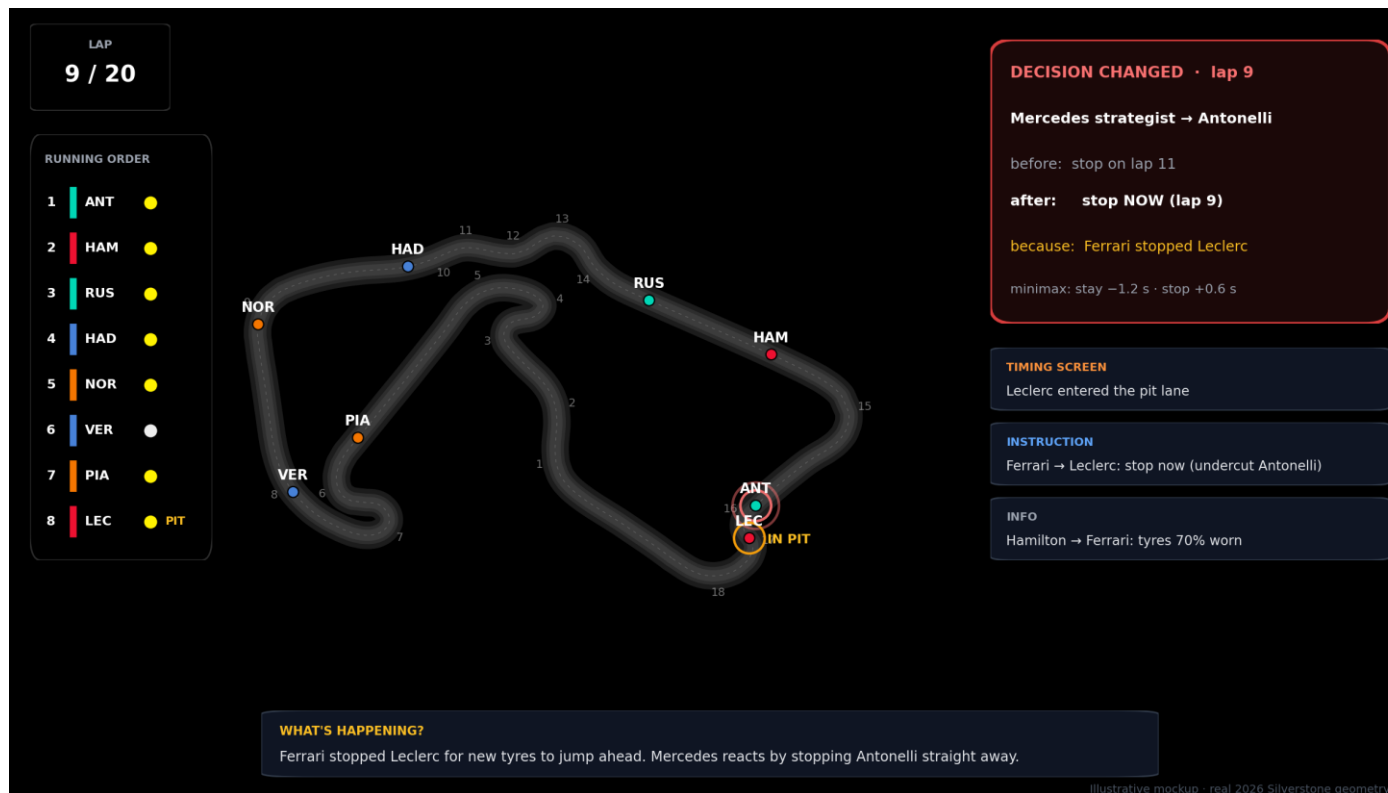


Figure 9. The track map panel (illustrative values) on the real 2026 Silverstone geometry

Element on the map panel	What it shows
Track and cars	Silverstone with corner numbers; each car a coloured dot with its code. The drawing code is copied from the F1-Telemetry-Analysis TrackMap component (copied, not linked)
Lap counter (top left)	Current lap / 20
Running order (left)	Position, team colour, driver code, tyre colour, 'PIT' while in the pit lane
Rings on cars	Amber ring: in the pit lane. Pulsing red ring: this car's plan just changed
Banner (top centre)	SAFETY CAR in yellow or RAIN in blue while active; blue tint on the map during rain
Decision changed card (top right)	Team → driver, before, after, because, and the search result behind it; stays about 10 seconds, newer cards push older ones down
Message notifications (right)	The latest messages in plain English with their type; fade after about 6 seconds (the terminal keeps the full log)
What's happening? (bottom)	One plain sentence explaining the latest event for the audience

9.4 The terminal

```
[lap 9] #40 Ferrari strategist -> Leclerc      INSTRUCTION Stop now (undercut Antonelli)
[lap 9] #41 Timing screen -> everyone          EVENT      Leclerc entered the pit lane
[lap 9] #42 Mercedes strategist -> Antonelli  INSTRUCTION Stop now to cover Leclerc
[lap 9] #43 Antonelli -> Mercedes strategist  REPLY      OK, boxing
[lap 9] DECISION CHANGED Mercedes / ANT: stop lap 11 -> lap 9 (because #41)
>
```

10. Commands

Commands are typed at the `>` prompt in terminal 1. Driver names can be given as a code or a surname in any case (`ant`, `ANT`, `antonelli`). A wrong command or name prints a hint and never stops the race.

Command	What it does
<code>start</code>	Start the race from the grid
<code>pause / resume</code>	Freeze and continue the race (useful while explaining a card)
<code>speed <1-20></code>	Race speed multiplier; default 10 (one lap ≈ 10 seconds)
<code>safety_car</code>	Deploy the Safety Car now, for 3 laps
<code>rain</code>	Start rain now; it stays wet until the end
<code>status</code>	Print the running order with tyre, tyre age, stops and each car's next planned stop
<code>plan <driver></code>	Print the car's current A* plan and how many nodes A* and UCS expanded to find it
<code>why <driver></code>	Explain the car's latest decision change: before, after, cause, and the A* or minimax numbers
<code>restart [seed]</code>	New race from the grid; the same seed repeats a race exactly, another seed gives a different race
<code>help</code>	List the commands
<code>quit</code>	Stop the backend

Example output (format; numbers come from the live race)

```
> plan NOR
Norris (McLaren) - lap 6 - Medium, 6 laps old
Best plan (A*): stay out to lap 11 -> pit for Hard -> finish          predicted race time 31:26.4
Search: A* expanded 214 nodes in 3 ms | UCS expanded 1,902 nodes in 24 ms (same cost)

> why ANT
Lap 9 - Mercedes strategist changed Antonelli's plan
  before:  pit on lap 11 for Hard
  after:    pit now (lap 9) for Hard
  because:  #41 Timing screen: 'Leclerc entered the pit lane'
  minimax: STAY -> 1.2 s behind Leclerc  PIT -> 0.6 s ahead   (1 leaf pruned)

> status
Lap 9/20 - green flag - dry
P Driver Team Tyre Age Stops Next stop
1 ANT Mercedes Medium 9 0 lap 9 (Hard)
2 HAM Ferrari Medium 9 0 lap 10 (Hard)
```

11. Demo script (about 7 minutes)

#	Laps	Type / do	Audience sees	Point out
0	–	<code>python run.py, npm run dev</code> , open the page	Silverstone, 8 cars on the real 2026 grid	12 agents: 4 strategists, 8 drivers; how to read the map
1	1–4	<code>start · plan NOR</code>	Plan notifications; terminal prints NOR's plan and A* vs UCS node counts	PEAS in action; why A*
2	4–8	(watch)	Hamilton reports worn tyres, then a red card 'lap 12 → lap 9 · because: Hamilton's report'	A driver changed the strategist's decision
3	8–11	(watch) · <code>why ANT</code>	Ferrari stops Leclerc, Mercedes reacts; card 'because: Ferrari stopped Leclerc'; terminal prints minimax values	A rival changed the decision; undercut; hidden tyres
4	~12	<code>safety_car</code>	Yellow banner, several cards at once, teammates stopping one behind the other	Sudden event: all teams replan; shared pit box. The real race had the same thing on laps 47–48
5	~15	<code>rain</code>	Blue tint, drivers report, teams switch to Wet tyres on different laps, order changes	Driver reports + event decide the timing
6	20	<code>speed 20</code>	Final order and a summary: decision changes by cause	Evidence of interaction; compare with the real result

- **Let the examiner choose** when to type `safety_car` and `rain`. The agents react whenever it happens, which covers the 'environmental shock' criterion.
- **Scenes 2 and 3 happen on their own** with seed 42: Hamilton has HIGH hidden wear in teams.json, and Antonelli and Leclerc start 1st and 2nd, which naturally creates the duel.
- **Backup: restart 42** repeats the rehearsed race exactly; keep a screen recording of a full run on the laptop.

12. Tools and project structure**12.1 Tools**

Tool	What it does in the project	Why
Python 3.12	Race engine, agents, backend	One language for all logic
Mesa 3	Agent-based modelling framework: agent classes, step order, seeded random numbers	Industry-standard framework named in the rubric; light to use
heapq (built into Python)	Priority queue for A* and UCS	Search code stays short and readable

Tool	What it does in the project	Why
pandas, NumPy	Read data/ and do the lap-time maths	Simple, well known
FastAPI + uvicorn	Serves the track data and streams the race over a WebSocket	Same backend framework as the existing F1 app
rich + prompt_toolkit	Coloured terminal log and a command prompt that is not broken by printed messages	Clean terminal during the demo
Next.js 16, React 19, Tailwind 4, framer-motion	The track map page, its overlays and notification animations	Same stack and versions as the existing F1 app, so the map code and look carry over
pytest	Tests	Shows the code is checked
FastF1, SciPy (tools/ only)	One-off data preparation	Not needed to run the demo

Not used: LLM agent frameworks such as CrewAI (slow, unpredictable, not search); ROS (robot middleware, heavy for an in-process simulation); game engines such as Pygame (would replace the existing track map instead of reusing it).

12.2 Folder structure

F1-Strategy-MAS/	
├── README.md	
├── requirements.txt	mesa, fastapi, uvicorn, pandas, numpy, rich, prompt_toolkit, pytest
├── run.py	starts the backend and the command prompt
├── data/	READY: track, corners, laps, summary, teams.json, race_model.json
├── tools/	READY: extract_race_data.py, fit_model.py, requirements.txt
├── sim/	
│ ├── model.py	reads race_model.json; lap-time and tyre-wear rules
│ ├── track.py	track and corners; car position on the map; the 3 straights
│ ├── race.py	RaceModel (Mesa): step order, pit stops, overtakes, Safety Car, rain
│ ├── messages.py	Message, MessageBoard (who can read what), DecisionChange
│ ├── driver.py	DriverAgent: condition-action rules
│ ├── strategist.py	StrategistAgent: belief update, A*, minimax, instructions
│ └── search.py	A*, UCS, minimax with alpha-beta, belief-state update
├── server/	
│ ├── api.py	FastAPI: GET /track, WebSocket /ws
│ └── console.py	the terminal commands
├── web/	Next.js app with one page
│ └── src/	
│ ├── app/page.tsx	full-screen track map panel
│ ├── components/RaceMap.tsx	canvas drawing (from TrackMap.tsx) + overlays
│ ├── components/Notifications.tsx	decision cards and message toasts
│ └── lib/raceSocket.ts	WebSocket hook and types
├── tests/	test_search.py, test_race.py
└── docs/	this plan

13. Build plan

Phase	Work	Time	Done when	Status
0 · Data	Download the 2026 race, extract files, fit the race model	–	data/ complete; app needs nothing else	Done
1 · Race engine	model.py , track.py , race.py : cars move, laps, traffic, overtakes, pit stops, Safety Car, rain	1.5 days	A 20-lap race runs headless with fixed stops and prints lap-by-lap order	To do
2 · Agents and algorithms	messages.py , search.py , driver.py , strategist.py	2 days	Plans change because of driver reports, rival stops, Safety Car and rain, each with a cause	To do
3 · Backend and terminal	server/api.py , server/console.py , run.py	1 day	Every command in §10 works; a WebSocket client receives the race	To do

Phase	Work	Time	Done when	Status
4 · Track map page	Next.js app: RaceMap, Notifications, raceSocket	1.5 days	Cars move smoothly; cards and notifications appear; Safety Car and rain banners show	To do
5 · Tune, test, rehearse	Seed 42 scenes, tests, backup video, slides	1 day	The §11 script runs three times in a row without problems	To do

Splitting the work for three people: person A does phase 1, person B does phase 2 (after agreeing the interfaces in `messages.py` with A), person C does phases 3 and 4. Everyone does phase 5 together.

14. Tests

Test	Checks
A* is optimal	On 20 random starting states, A* and UCS return the same plan cost, and A* expands no more nodes
Heuristic is admissible	On short races (3–4 laps), h never exceeds the true cheapest cost found by UCS
Worked example	The Figure 2 case gives 305.69 s, pit for Soft, with 5 A* expansions
Minimax	The Figure 3 case returns PIT with value +0.6 and prunes the last leaf
Belief update	A rival with HIGH wear is narrowed down to {HIGH} within a few clean laps
Privacy	A strategist never receives another team's radio messages
Race runs	20 different seeds with Safety Car and rain at random laps all finish without errors

15. Preparing and defending

15.1 Slides (6)

#	Slide	Content	Rubric
1	Problem & PEAS	F1 in 60 seconds (6 terms); problem in one sentence; PEAS table; 3 rows of the impact matrix	P1 PEAS
2	Environment & agents	Classification table; cooperative vs competitive; agent types; state-space numbers	P1 Environment
3	Algorithms & why	Decision → algorithm table; A* example (Fig. 2); minimax tree (Fig. 3); 'why not' in 4 short lines	P1 Algorithms
4	Data, tools & structure	Real 2026 race facts; data vs hand-set numbers; architecture (Fig. 8); folder tree	P2 Tools, Code
5	Interactions	The 4 interaction diagrams (Figs. 4–7) as thumbnails; request → reply rule; decision-change record	P2 Interaction
6	Demo	Scene list and commands; 'you choose when the Safety Car and rain happen'; then the live demo	P2 Demo

15.2 Preparation checklist

- Every member can explain the PEAS table and the environment classification without notes.
- Every member can work through the A* example (Fig. 2) and the minimax tree (Fig. 3) on a whiteboard, including why h is admissible and which leaf alpha-beta skips.

- Know the data facts: 2026 British GP, dry, 52 laps, real grid, 21 s pit loss from 24 stops, which numbers are hand-set and why.
- Rehearse the §11 script three times; decide who talks and who types.
- Presentation laptop: `pip install` and `npm install` already done, ports 8000 and 3000 free, page readable on the projector, backup video on the desktop.

15.3 Q&A

Question	Answer
Why A* and not BFS?	BFS finds the plan with the fewest steps. Our steps cost different times (a stop ~21 s, a lap ~95 s), so we need a cost-optimal search.
Why A* and not UCS? Both are optimal.	Same answer, less work. In our example A* expands 5 nodes and UCS 18, because h points the search at promising plans. <code>plan</code> shows this live.
Prove the heuristic is admissible.	It is the cost of a relaxed problem (no tyre wear, fastest tyre, one stop). A relaxed problem is never more expensive than the real one.
Why not greedy best-first? It is faster.	It follows only h and ignores the time already spent, so it can return a slower plan.
Why minimax only for duels?	Minimax needs two players with opposite goals. Two cars fighting over one position fit that; four teams over 20 laps do not, and the tree would be astronomically large.
Why not Expectiminimax for the Safety Car chance?	It would add a chance node on every lap. We replan the moment a Safety Car happens, which gives the right decision for what actually occurred with a tiny tree.
What does alpha-beta save here?	Branches that cannot change the result, like the last leaf in Figure 3. The answer is always the same as plain minimax, with less work.
How do you handle not knowing rival tyres?	Belief states: we keep the set of wear levels that are still consistent with the lap times we observe (AIMA Ch. 4).
Is it really multi-agent?	Each agent has its own information and rules and acts only through messages or the track. The end summary counts how many decisions were changed by another agent.
Is this real data?	Yes: the 2026 British Grand Prix, a dry race. Base pace, fuel effect, Medium tyre wear and the 21 s pit loss are measured from it; values the race cannot give us are set by hand and labelled.
Why 20 laps, not 52?	To fit a live demo. Tyre wear and fuel are scaled by 52/20 so the strategy problem stays the same.
Does it need the internet or your other app?	No. All data is inside the project; the page and backend run locally.

16. Later: merging into the F1 app

- `sim/` has no web code, so it can move into the F1-Telemetry-Analysis backend and be exposed with a few FastAPI routes.
- `RaceMap.tsx` uses the same drawing code as `TrackMap.tsx` and the same Next.js version, so it can become a new tab in that app.
- With any race the app loads, `tools/fit_model.py` and the same A* planner and belief update can **predict** pit windows and undercut threats during replays.